

Summary of Performance Testing

The script which was used to automate the testing repeatedly over is added in the Appendix for reference.

I have also added log snippets during the perf. Test to show how the query is getting executed.

In Summary ,we can see the query executing below 10ms consistently.

Test Size	CUSIPs	Lending IDs	Run 1 Time (ms)	Run 2 Time (ms)	Run 3 Time (ms)	Run 4 Time (ms)	Avg Time (ms)	Docs Examined (Avg)	Keys Examined (Avg)	Docs Returned (Avg)
Small	100	100	2	3	2	2	2.5	185	272	12
Medium	500	500	7	8	7	7	7.75	872	1327	152
Large	1000	1000	10	9	8	8	9.75	1711	2629	434
XLarge	2000	2000	12	13	12	12	12.5	3504	5397	1051

Note on Averaging:

- **Time (ms):** The average is calculated from the four recorded execution times.
- **Docs Examined / Keys Examined / Docs Returned:** The average is calculated across the four runs.

Database: q_sl_approvals

Collection: validation_rules

Total Documents: 100000

Current Indexes on collection:

```
JavaScript
_id_ (Keys: {"_id": 1})

meta.isActive_1_cusipGroups.values_1 (Keys: {"meta.isActive": 1, "cusipGroups.values": 1})

meta.isActive_1_lendingIdGroups.values_1 (Keys: {"meta.isActive": 1, "lendingIdGroups.values": 1})

meta.isActive_1_rules.market_1 (Keys: {"meta.isActive": 1, "rules.market": 1})
```

Final Summary Table (Latest Run)

Test	CUSIPs	LendIDs	Time (ms)	Docs Examined	Keys Examined	Returned
Small	100	100	2	204	293	9
Medium	500	500	7	886	1340	153
Large	1000	1000	8	1852	2786	436
XLarge	2000	2000	11	3568	5464	1034

Best index based on Performance

JavaScript

```
db.validation_rules.createIndex({  
  "meta.isActive": 1,  
  "lendingIdGroups.values": 1  
})
```

Summary of Indexes Used:

The query consistently used the following index across all tests in all four runs:

- `meta.isActive_1_lendingIdGroups.values_1` (Keys: {"meta.isActive":1,"lendingIdGroups.values":1})

Query Plan Stage:

The query plan consistently showed the main stages as:

- **Stage:** `FETCH` {For getting the `cusipGroups.values` array element}
- **Input Stage:** `IXSCAN` {for getting Market}

I have attached the complete test results as a separate text file , showing performance during each execution.

NOTE : *The first few runs will be slower as this is when the working set is loaded from Disk to Memory. Once the working set is present in Memory the execution becomes faster. So you need to repeat the performance test multiple times.*

This is the summary from the very first run with higher execution times.

SUMMARY TABLE

Test	CUSIPs	LendIDs	Time (ms)	Docs Examined	Keys Examined	Returned	
Small	100	100	20	188	276	8	
Medium	500	500	33	940	1395	190	
Large	1000	1000	29	1808	2730	438	
XLarge	2000	2000	33	3517	5408	1048	

Same test repeated showed reduced execution times

SUMMARY TABLE

Test	CUSIPs	LendIDs	Time (ms)	Docs Examined	Keys Examined	Returned	
Small	100	100	4	188	280	13	
Medium	500	500	11	894	1352	170	
Large	1000	1000	13	1832	2745	466	
XLarge	2000	2000	16	3594	5461	1058	

Same test on the 3rd execution

SUMMARY TABLE

Test	CUSIPs	LendIDs	Time (ms)	Docs Examined	Keys Examined	Returned	
Small	100	100	2	204	293	9	
Medium	500	500	7	886	1340	153	
Large	1000	1000	8	1852	2786	436	
XLarge	2000	2000	11	3568	5464	1034	

Appendix

1. Performance Testing script (can be re-used with your own modifications)

JavaScript

// performance_test.js - Fixed version with proper error handling

```
const DB_NAME = "q_sl_approvals";  
const COLLECTION = "validation_rules";
```

```
db = db.getSiblingDB(DB_NAME);
```

```
print(`\n🔗 Connected to database: ${DB_NAME}`);  
print(`📁 Collection: ${COLLECTION}`);  
print(`📊 Total documents: ${db[COLLECTION].countDocuments({})}`);
```

// Helper function to extract execution stats from different explain formats

```
function extractStats(explainResult) {  
  let stats = null;  
  let winningPlan = null;  
  
  // Format 1: Aggregation with stages array (MongoDB 5.0+)  
  if (explainResult.stages && explainResult.stages[0]) {  
    if (explainResult.stages[0].$cursor) {  
      stats = explainResult.stages[0].$cursor.executionStats;  
      winningPlan = explainResult.stages[0].$cursor.queryPlanner.winningPlan;  
    }  
  }  
}
```

```

// Format 2: Direct executionStats (find queries or older MongoDB)
if (!stats && explainResult.executionStats) {
    stats = explainResult.executionStats;
    winningPlan = explainResult.queryPlanner ? explainResult.queryPlanner.winningPlan : null;
}

// Format 3: Nested in queryPlanner
if (!stats && explainResult.queryPlanner) {
    winningPlan = explainResult.queryPlanner.winningPlan;
    stats = explainResult.executionStats || {
        executionTimeMillis: 0,
        totalDocsExamined: 0,
        totalKeysExamined: 0,
        nReturned: 0
    };
}

// Format 4: Sharded cluster format
if (!stats && explainResult.shards) {
    const shardNames = Object.keys(explainResult.shards);
    if (shardNames.length > 0) {
        const firstShard = explainResult.shards[shardNames[0]];
        if (firstShard.stages && firstShard.stages[0].$cursor) {
            stats = firstShard.stages[0].$cursor.executionStats;
            winningPlan = firstShard.stages[0].$cursor.queryPlanner.winningPlan;
        }
    }
}

return { stats, winningPlan };

```

```

}

// Performance test function
function runPerformanceTest(numCusips, numLendingIds, markets) {
  markets = markets || ["JPN", "USA", "GBR", "DEU", "AUS"];

  // Generate CUSIP values
  const cusipValues = [];
  for (let i = 0; i < numCusips; i++) {
    cusipValues.push(`CUSIP_${String(Math.floor(Math.random() * 9999)).padStart(4, '0')}}`);
  }
  cusipValues.push("*");

  // Generate Lending ID values
  const lendingIdValues = [];
  for (let i = 0; i < numLendingIds; i++) {
    lendingIdValues.push(`LEND_${String(Math.floor(Math.random() * 999999)).padStart(6, '0')}}`);
  }

  const matchStage = {
    "meta.isActive": true,
    "markets": { "$in": markets },
    "cusipGroups.values": { "$in": cusipValues },
    "lendingIdGroups.values": { "$in": lendingIdValues }
  };

  const pipeline = [{ $match: matchStage }];

  print(`\n🕒 Running test with ${numCusips} CUSIPs and ${numLendingIds} Lending IDs...`);

  // Get explain output

```



```

let explainResult;
try {
  explainResult = db[COLLECTION].aggregate(pipeline).explain("executionStats");
} catch (e) {
  print(`❌ Error running explain: ${e.message}`);
  return null;
}

// Extract stats using helper function
const { stats, winningPlan } = extractStats(explainResult);

if (!stats) {
  print(`\n⚠️ Could not extract execution stats. Raw explain output:`);
  printjson(explainResult);
  return explainResult;
}

// Display results
print(`\n=====`);
print(`| PERFORMANCE TEST RESULTS |`);
print(`|=====|`);
print(`| Test Parameters: |`);
print(`| CUSIP values in $in: ${String(cusipValues.length).padStart(8, ' ')} |`);
print(`| Lending ID values in $in: ${String(lendingIdValues.length).padStart(8, ' ')} |`);
print(`|`);
print(`| Markets: ${String(markets.length).padStart(8, ' ')} |`);
print(`|=====|`);
print(`| Execution Stats: |`);
print(`| Execution time: ${String((stats.executionTimeMillis || 0) + ' ms').padStart(12, ' ')} |`);
print(`|`);

```

```

    print(`|| Documents examined:      ${String(stats.totalDocsExamined || 0).padStart(12, ' ')}
||`);
    print(`|| Keys examined:          ${String(stats.totalKeysExamined || 0).padStart(12, ' ')}
||`);
    print(`|| Documents returned:      ${String(stats.nReturned || 0).padStart(12, ' ')} ||`);
    print(`||=====||`);

    // Display winning plan info
    if (winningPlan) {
        print(`|| Query Plan:                                     ||`);
        print(`|| Stage: ${String(winningPlan.stage || 'N/A').padEnd(48, ' ')} ||`);

        if (winningPlan.inputStage) {
            print(`|| Input Stage: ${String(winningPlan.inputStage.stage || 'N/A').padEnd(42, ' ')} ||`);
            if (winningPlan.inputStage.indexName) {
                print(`|| Index Used: ${String(winningPlan.inputStage.indexName || 'N/A').padEnd(43, ' ')} ||`);
            }
            if (winningPlan.inputStage.inputStage) {
                print(`|| Inner Stage: ${String(winningPlan.inputStage.inputStage.stage || 'N/A').padEnd(42, ' ')} ||`);
                if (winningPlan.inputStage.inputStage.indexName) {
                    print(`|| Index Used: ${String(winningPlan.inputStage.inputStage.indexName).padEnd(43, ' ')} ||`);
                }
            }
        }
    }

    print(`||=====||`);

    return {
        stats: stats,
        winningPlan: winningPlan,
    }
}

```

```

    explainResult: explainResult
  };
}

// Function to run all tests
function runAllTests() {
  print("\n");
  print("=".repeat(60));
  print("      🚀 MONGODB QUERY PERFORMANCE TEST SUITE");
  print("=".repeat(60));

  // Show current indexes
  print("\n📖 Current Indexes on collection:");
  db[COLLECTION].getIndexes().forEach((idx, i) => {
    print(`    ${i + 1}. ${idx.name}`);
    print(`        Keys: ${JSON.stringify(idx.key)}`);
  });

  // Define test cases
  const testCases = [
    { cusips: 100, lendingIds: 100, name: "Small" },
    { cusips: 500, lendingIds: 500, name: "Medium" },
    { cusips: 1000, lendingIds: 1000, name: "Large" },
    { cusips: 2000, lendingIds: 2000, name: "XLarge" }
  ];

  const results = [];

  testCases.forEach((test, index) => {
    print(`\n${"—"}.repeat(60)`)
    print(`📊 Test ${index + 1}/${testCases.length}: ${test.name}`);
  });
}

```

```

const result = runPerformanceTest(test.cusips, test.lendingIds);

if (result && result.stats) {
  results.push({
    name: test.name,
    cusips: test.cusips,
    lendingIds: test.lendingIds,
    timeMs: result.stats.executionTimeMillis || 0,
    docsExamined: result.stats.totalDocsExamined || 0,
    keysExamined: result.stats.totalKeysExamined || 0,
    returned: result.stats.nReturned || 0
  });
}
});

// Summary table
if (results.length > 0) {
  print(`\n\n${"=".repeat(90)}`);
  print("                                SUMMARY TABLE");
  print("=".repeat(90));
  print("| Test      | CUSIPs | LendIDs | Time (ms) | Docs Examined | Keys Examined | Returned |");
  print("-".repeat(90));

  results.forEach(r => {
    const row = `| ${r.name.padEnd(8)} | ${String(r.cusips).padStart(6)} | ${String(r.lendingIds).padStart(7)}
| ${String(r.timeMs).padStart(9)} | ${String(r.docsExamined).padStart(13)} |
${String(r.keysExamined).padStart(13)} | ${String(r.returned).padStart(8)} |`;
    print(row);
  });
}

```

```

    print("=".repeat(90));
  }

  print("\n✅ All tests completed!\n");

  return results;
}

// Also provide a simple single test function
function quickTest(numCusips, numLendingIds) {
  numCusips = numCusips || 1000;
  numLendingIds = numLendingIds || 1000;
  return runPerformanceTest(numCusips, numLendingIds);
}

// Run the full test suite
runAllTests();

```

2. Log snippets with Actual Query plan and execution.

JSON

```

// 1st Query
{"t":{"$date":"2026-03-13T12:22:43.071+00:00"},"s":"I", "c":"COMMAND", "id":"51803", "ctx":"conn47","msg":"Slow query","attr":{"type":"command","isFromUserConnection":true,"ns":"q_sl_approvals.validation_rules","collectionType":"normal","appName":"mongosh 2.5.9","queryShapeHash":"4C4783B538E9BD101ABA8259543F5E010FD7806750007B6B8515487BD3964B95"},"command":{"explain":{

```

```
"aggregate":"validation_rules","pipeline":[{"$match":{"meta.isActive":true,"markets":{"$in":["JPN","USA","GBR","D
EU","AUS"]},"cusipGroups.values":{"$in":["CUSIP_9023","CUSIP_9910","CUSIP_3777","CUSIP_3637","CUSIP_3005","CUSIP_
1925","CUSIP_8683","CUSIP_2805","CUSIP_3946","CUSIP_3768","CUSIP_5713","CUSIP_5995","CUSIP_5161","CUSIP_3977","CU
SIP_2899","CUSIP_9454","CUSIP_6244","CUSIP_4645","CUSIP_6961","CUSIP_1146","CUSIP_7178","CUSIP_8079","CUSIP_9834"
,"CUSIP_0456","CUSIP_5948","CUSIP_7721","CUSIP_3690","CUSIP_5251","CUSIP_7062","CUSIP_2784","CUSIP_2646","CUSIP_9
100","CUSIP_5947","CUSIP_6335","CUSIP_8130","CUSIP_7956","CUSIP_9370","CUSIP_4181","CUSIP_0149","CUSIP_4429","CUS
IP_4635","CUSIP_1700","CUSIP_3652","CUSIP_9024","CUSIP_4783","CUSIP_7235","CUSIP_8690","CUSIP_2122","CUSIP_4976",
"CUSIP_2035","CUSIP_6856","CUSIP_1132","CUSIP_0202","CUSIP_4119","CUSIP_3754","CUSIP_7084","CUSIP_1286","CUSIP_27
80","CUSIP_1558","CUSIP_6109","CUSIP_7956","CUSIP_1055","CUSIP_9726","CUSIP_2033","CUSIP_4486","CUSIP_2159","CUST
IP_1580","CUSIP_4306","CUSIP_3318","CUSIP_6543","CUSIP_7131","CUSIP_1099","CUSIP_4470","CUSIP_3392","CUSIP_1971","
CUSIP_3855","CUSIP_1174","CUSIP_7819","CUSIP_8020","CUSIP_0407","CUSIP_3759","CUSIP_7106","CUSIP_5054","CUSIP_847
5","CUSIP_7243","CUSIP_5108","CUSIP_3456","CUSIP_9760","CUSIP_5654","CUSIP_7466","CUSIP_9039","CUSIP_5492","CUSIP
_7623","CUSIP_0193","CUSIP_9091","CUSIP_6304","CUSIP_2293","CUSIP_6467","CUSIP_4744","CUSIP_4500","CUSIP_5642","C
USIP_3333","CUSIP_9825","CUSIP_3960","CUSIP_2942","CUSIP_8089","CUSIP_7585","CUSIP_6251","CUSIP_5670","CUSIP_4645
","CUSIP_9790","CUSIP_6749","CUSIP_4211","CUSIP_2804","CUSIP_1320","CUSIP_2909","CUSIP_0459","CUSIP_2904","CUSIP_
7849","CUSIP_3453","CUSIP_0949","CUSIP_8791","CUSIP_7129","CUSIP_3060","CUSIP_2601","CUSIP_4036","CUSIP_3690","CU
SIP_7607","CUSIP_1127","CUSIP_1712","CUSIP_3604","CUSIP_5223","CUSIP_2642","CUSIP_2724","CUSIP_5473","CUSIP_4208"
,"CUSIP_3460","CUSIP_3998","CUSIP_1432","CUSIP_3870","CUSIP_5474","CUSIP_0072","CUSIP_4397","CUSIP_0370","CUSIP_2
118","CUSIP_9485","CUSIP_8662","CUSIP_3837","CUSIP_5217","CUSIP_0726","CUSIP_4052","CUSIP_2346","CUSIP_6820","CUS
IP_6998","CUSIP_0317","CUSIP_2557","CUSIP_4504","CUSIP_0465","CUSIP_7363","CUSIP_1731","CUSIP_7216","CUSIP_6446",
"CUSIP_0165","CUSIP_5575","CUSIP_2144","CUSIP_0346","CUSIP_7914","CUSIP_5214","CUSIP_4033","CUSIP_5479","CUSIP_81
39","CUSIP_2208","CUSIP_9037","CUSIP_0564","CUSIP_2907","CUSIP_6382","CUSIP_7768","CUSIP_0451","CUSIP_4133","CUST
IP_8463","CUSIP_4829","CUSIP_0605","CUSIP_6191","CUSIP_3285","CUSIP_0350","CUSIP_7846","CUSIP_6316","CUSIP_9606","
CUSIP_9255","CUSIP_3029","CUSIP_1514","CUSIP_4248","CUSIP_6547","CUSIP_8387","CUSIP_8310","CUSIP_3239","CUSIP_413
3","CUSIP_6825","CUSIP_0641","CUSIP_9970","CUSIP_7972","CUSIP_6909","CUSIP_7884","CUSIP_4415","CUSIP_7310","CUSIP
_4351","CUSIP_8194","CUSIP_1684","CUSIP_9997","CUSIP_2976","CUSIP_1507","CUSIP_8696","CUSIP_8350","CUSIP_9158","C
USIP_8164","CUSIP_6858","CUSIP_4584","CUSIP_8909","CUSIP_9029","CUSIP_0451","CUSIP_9913","CUSIP_0225","CUSIP_3992
","CUSIP_3902","CUSIP_6120","CUSIP_1655","CUSIP_2028","CUSIP_2783","CUSIP_8158","CUSIP_3954","CUSIP_6866","CUSIP_
1807","CUSIP_6258","CUSIP_8756","CUSIP_7093","CUSIP_6924","CUSIP_5215","CUSIP_6690","CUSIP_1434","CUSIP_9736","CU
SIP_2432","CUSIP_6861","CUSIP_2551","CUSIP_2633","CUSIP_1791","CUSIP_6485","CUSIP_3239","CUSIP_9830","CUSIP_3940"
,"CUSIP_1732","CUSIP_2492","CUSIP_4670","CUSIP_4795","CUSIP_8595","CUSIP_4192","CUSIP_9105","CUSIP_8046","CUSIP_3
```

769", "CUSIP_4026", "CUSIP_2351", "CUSIP_5169", "CUSIP_3884", "CUSIP_4395", "CUSIP_5474", "CUSIP_2036", "CUSIP_9602", "CUSIP_9761", "CUSIP_2714", "CUSIP_7592", "CUSIP_8316", "CUSIP_9524", "CUSIP_1156", "CUSIP_5266", "CUSIP_1662", "CUSIP_6543", "CUSIP_0908", "CUSIP_2058", "CUSIP_5125", "CUSIP_6417", "CUSIP_9380", "CUSIP_8162", "CUSIP_0026", "CUSIP_5745", "CUSIP_3129", "CUSIP_6829", "CUSIP_2100", "CUSIP_4902", "CUSIP_2375", "CUSIP_1308", "CUSIP_8050", "CUSIP_0616", "CUSIP_6534", "CUSIP_4800", "CUSIP_2947", "CUSIP_5043", "CUSIP_4860", "CUSIP_0646", "CUSIP_2615", "CUSIP_6399", "CUSIP_1866", "CUSIP_7938", "CUSIP_5412", "CUSIP_1807", "CUSIP_2630", "CUSIP_5804", "CUSIP_2189", "CUSIP_7493", "CUSIP_6186", "CUSIP_6817", "CUSIP_2018", "CUSIP_2442", "CUSIP_6366", "CUSIP_5914", "CUSIP_4802", "CUSIP_8285", "CUSIP_6281", "CUSIP_0606", "CUSIP_9874", "CUSIP_6711", "CUSIP_5688", "CUSIP_7335", "CUSIP_6488", "CUSIP_5851", "CUSIP_2501", "CUSIP_4367", "CUSIP_8174", "CUSIP_7521", "CUSIP_5108", "CUSIP_6757", "CUSIP_6398", "CUSIP_9027", "CUSIP_4082", "CUSIP_6790", "CUSIP_9838", "CUSIP_2103", "CUSIP_0085", "CUSIP_9920", "CUSIP_8974", "CUSIP_3166", "CUSIP_4967", "CUSIP_1373", "CUSIP_7393", "CUSIP_0527", "CUSIP_3556", "CUSIP_4417", "CUSIP_1332", "CUSIP_9047", "CUSIP_2597", "CUSIP_9858", "CUSIP_1922", "CUSIP_1616", "CUSIP_0697", "CUSIP_5343", "CUSIP_0971", "CUSIP_3958", "CUSIP_6309", "CUSIP_2111", "CUSIP_1995", "CUSIP_4042", "CUSIP_5150", "CUSIP_8175", "CUSIP_9398", "CUSIP_5025", "CUSIP_3146", "CUSIP_0829", "CUSIP_9407", "CUSIP_8198", "CUSIP_6696", "CUSIP_2317", "CUSIP_0295", "CUSIP_6506", "CUSIP_2673", "CUSIP_7165", "CUSIP_8394", "CUSIP_4576", "CUSIP_0238", "CUSIP_9188", "CUSIP_3643", "CUSIP_5206", "CUSIP_3545", "CUSIP_5509", "CUSIP_6432", "CUSIP_2995", "CUSIP_4347", "CUSIP_4652", "CUSIP_9510", "CUSIP_4165", "CUSIP_9010", "CUSIP_0756", "CUSIP_9389", "CUSIP_1402", "CUSIP_4847", "CUSIP_5434", "CUSIP_2617", "CUSIP_7181", "CUSIP_3072", "CUSIP_4305", "CUSIP_2356", "CUSIP_1335", "CUSIP_4293", "CUSIP_2478", "CUSIP_4634", "CUSIP_8625", "CUSIP_8853", "CUSIP_2945", "CUSIP_9743", "CUSIP_1306", "CUSIP_6850", "CUSIP_6051", "CUSIP_3023", "CUSIP_0587", "CUSIP_4155", "CUSIP_5093", "CUSIP_8801", "CUSIP_0128", "CUSIP_6829", "CUSIP_2850", "CUSIP_4524", "CUSIP_1843", "CUSIP_7855", "CUSIP_2436", "CUSIP_9579", "CUSIP_5748", "CUSIP_5060", "CUSIP_9190", "CUSIP_1475", "CUSIP_5492", "CUSIP_7705", "CUSIP_6057", "CUSIP_2863", "CUSIP_2142", "CUSIP_7537", "CUSIP_9619", "CUSIP_3966", "CUSIP_7373", "CUSIP_5566", "CUSIP_9598", "CUSIP_8071", "CUSIP_1506", "CUSIP_3034", "CUSIP_3754", "CUSIP_1030", "CUSIP_0138", "CUSIP_2371", "CUSIP_3582", "CUSIP_3311", "CUSIP_9158", "CUSIP_0992", "CUSIP_7732", "CUSIP_3670", "CUSIP_1147", "CUSIP_6798", "CUSIP_3221", "CUSIP_1926", "CUSIP_5321", "CUSIP_1525", "CUSIP_0418", "CUSIP_2630", "CUSIP_6404", "CUSIP_4041", "CUSIP_0143", "CUSIP_2782", "CUSIP_8641", "CUSIP_4900", "CUSIP_6340", "CUSIP_6845", "CUSIP_9368", "CUSIP_5371", "CUSIP_7182", "CUSIP_5378", "CUSIP_5252", "CUSIP_2996", "CUSIP_5761", "CUSIP_0653", "CUSIP_5507", "CUSIP_2693", "CUSIP_1762", "CUSIP_1917", "CUSIP_1031", "CUSIP_9814", "CUSIP_6815", "CUSIP_1410", "CUSIP_3534", "CUSIP_1162", "CUSIP_8423", "CUSIP_6172", "CUSIP_7562", "CUSIP_5162", "CUSIP_9824", "CUSIP_2697", "CUSIP_7151", "CUSIP_1933", "CUSIP_9695", "CUSIP_8190", "CUSIP_5278", "CUSIP_9660", "CUSIP_1824", "CUSIP_4632", "CUSIP_8965", "CUSIP_6022", "*" }}, "lendingIdGroups.values": { "\$in": ["LEND_648195", "LEND_633707", "LEND_224786", "LEND_455603", "LEND_441594", "LEND_984728", "LEND_366276", "LEND_543542", "LEND_482039", "LEND_791839", "LEND_074974", "LEND_682805", "LEND_621470", "LEND_811221", "LEND_280846", "LEND_027730", "LEND_265604", "LEND_715416", "LEND_301762", "LEND_843528", "LEND_774715", "LEND_970833", "LEND

```
_417518", "LEND_518424", "LEND_704465", "LEND_289584", "LEND_733736", "LEND_770721", "LEND_664535", "LEND_390020", "LEND_212608", "LEND_263964", "LEND_539424", "LEND_792952", "LEND_724135", "LEND_569004", "LEND_406663", "LEND_159686", "LEND_104807", "LEND_408500", "LEND_113673", "LEND_836753", "LEND_667643", "LEND_145554", "LEND_826703", "LEND_533679", "LEND_979958", "LEND_994904", "LEND_029564", "LEND_411209", "LEND_302403", "LEND_404695", "LEND_824645", "LEND_609180", "LEND_108663", "LEND_081420", "LEND_667151", "LEND_429888", "LEND_189665", "LEND_943619", "LEND_289303", "LEND_066166", "LEND_207670", "LEND_420928", "LEND_689101", "LEND_056959", "LEND_559223", "LEND_737577", "LEND_506772", "LEND_095585", "LEND_405155", "LEND_224476", "LEND_793575", "LEND_900862", "LEND_254492", "LEND_317703", "LEND_830551", "LEND_777497", "LEND_695220", "LEND_425448", "LEND_475210", "LEND_957314", "LEND_600678", "LEND_187861", "LEND_644447", "LEND_720106", "LEND_250452", "LEND_733590", "LEND_388671", "LEND_156055", "LEND_785651", "LEND_252557", "LEND_811042", "LEND_959946", "LEND_018564", "LEND_371757", "LEND_289193", "LEND_637075", "LEND_894954", "LEND_370095", "LEND_512199", "LEND_736021", "LEND_180188", "LEND_674738", "LEND_485038", "LEND_768896", "LEND_519114", "LEND_513421", "LEND_797836", "LEND_072298", "LEND_539022", "LEND_601809", "LEND_417191", "LEND_811010", "LEND_148282", "LEND_153078", "LEND_867142", "LEND_327472", "LEND_038339", "LEND_295860", "LEND_343406", "LEND_855846", "LEND_211977", "LEND_001242", "LEND_547273", "LEND_577508", "LEND_295999", "LEND_899012", "LEND_761711", "LEND_217059", "LEND_117830", "LEND_396030", "LEND_817901", "LEND_754080", "LEND_057213", "LEND_041403", "LEND_169126", "LEND_640231", "LEND_971300", "LEND_548693", "LEND_794599", "LEND_838717", "LEND_552084", "LEND_655332", "LEND_445867", "LEND_715561", "LEND_398403", "LEND_783718", "LEND_831999", "LEND_629977", "LEND_973205", "LEND_591901", "LEND_791399", "LEND_641198", "LEND_057698", "LEND_639247", "LEND_469695", "LEND_934894", "LEND_839120", "LEND_672095", "LEND_876124", "LEND_712994", "LEND_664621", "LEND_897111", "LEND_618876", "LEND_568573", "LEND_893606", "LEND_453425", "LEND_637781", "LEND_376101", "LEND_278502", "LEND_337510", "LEND_355766", "LEND_604468", "LEND_637870", "LEND_731740", "LEND_018671", "LEND_398392", "LEND_222528", "LEND_795099", "LEND_553406", "LEND_493308", "LEND_592710", "LEND_331925", "LEND_716749", "LEND_519477", "LEND_261214", "LEND_171100", "LEND_542102", "LEND_936374", "LEND_962301", "LEND_509473", "LEND_208929", "LEND_278831", "LEND_274321", "LEND_116073", "LEND_690542", "LEND_233755", "LEND_560074", "LEND_904619", "LEND_607533", "LEND_001772", "LEND_237016", "LEND_278559", "LEND_994513", "LEND_571306", "LEND_293592", "LEND_543916", "LEND_817268", "LEND_264938", "LEND_405415", "LEND_204257", "LEND_030146", "LEND_221008", "LEND_505370", "LEND_674342", "LEND_094347", "LEND_904131", "LEND_431016", "LEND_996531", "LEND_689131", "LEND_061870", "LEND_840459", "LEND_728910", "LEND_542114", "LEND_857911", "LEND_639268", "LEND_361444", "LEND_759127", "LEND_348503", "LEND_518352", "LEND_160081", "LEND_570291", "LEND_728712", "LEND_975742", "LEND_202277", "LEND_563299", "LEND_479313", "LEND_073061", "LEND_444073", "LEND_006461", "LEND_147299", "LEND_076949", "LEND_497939", "LEND_754718", "LEND_584445", "LEND_092169", "LEND_613353", "LEND_883706", "LEND_871108", "LEND_959266", "LEND_377518"]]]]]]]], "planSummary": "IXSCAN { meta.isActive: 1, lendingIdGroups.values: 1 }", "planningTimeMicros": 6423, "numYields": 0, "planCacheShapeHash": "2D899393", "queryHash": "2D899393", "planCacheKey":
```



```
"F1CE8BF5", "queryFramework": "classic", "reslen": 152706, "locks": {"Global": {"acquireCount": {"r": 1}}}, "readConcern": {"level": "local", "provenance": "implicitDefault"}, "storage": {}, "remote": "127.0.0.1:53598", "protocol": "op_msg", "queues": {"ingress": {"admissions": 1}, "execution": {"admissions": 2}}, "workingMillis": 7, "durationMillis": 7, "truncated": {"command": {"truncated": {"explain": {"truncated": {"pipeline": {"truncated": {"0": {"truncated": {"$match": {"truncated": {"lendingIdGroups.values": {"truncated": {"$in": {"truncated": {"252": {"type": "string", "size": 16}}, "omitted": 248}}}}}}}}, "omitted": 1}}, "omitted": 5}}, "size": {"command": 20797}}
```

// 2nd Query

```
{"t": {"$date": "2026-03-13T12:25:20.069+00:00"}, "s": "I", "c": "COMMAND", "id": 51803, "ctx": "conn50", "msg": "Slow query", "attr": {"type": "command", "isFromUserConnection": true, "ns": "q_sl_approvals.validation_rules", "collectionType": "normal", "appName": "mongosh"}, "queryShapeHash": "4C4783B538E9BD101ABA8259543F5E010FD7806750007B6B8515487BD3964B95", "command": {"explain": {"aggregate": "validation_rules", "pipeline": [{"$match": {"meta.isActive": true, "markets": {"$in": ["JPN", "USA", "GBR", "DEU", "AUS"]}, "cusipGroups.values": {"$in": ["CUSIP_4572", "CUSIP_8757", "CUSIP_1878", "CUSIP_1430", "CUSIP_3454", "CUSIP_4577", "CUSIP_3304", "CUSIP_4947", "CUSIP_0794", "CUSIP_6775", "CUSIP_1745", "CUSIP_3897", "CUSIP_9287", "CUSIP_0196", "CUSIP_3729", "CUSIP_8808", "CUSIP_4741", "CUSIP_1722", "CUSIP_3489", "CUSIP_8824", "CUSIP_7397", "CUSIP_0738", "CUSIP_3516", "CUSIP_8001", "CUSIP_4019", "CUSIP_4138", "CUSIP_8414", "CUSIP_6600", "CUSIP_3816", "CUSIP_7492", "CUSIP_5015", "CUSIP_9086", "CUSIP_5070", "CUSIP_6555", "CUSIP_3549", "CUSIP_4997", "CUSIP_0892", "CUSIP_5728", "CUSIP_3080", "CUSIP_2968", "CUSIP_2502", "CUSIP_4925", "CUSIP_4949", "CUSIP_4237", "CUSIP_1864", "CUSIP_8107", "CUSIP_5786", "CUSIP_5465", "CUSIP_3364", "CUSIP_4189", "CUSIP_7712", "CUSIP_1196", "CUSIP_1169", "CUSIP_8467", "CUSIP_8049", "CUSIP_0390", "CUSIP_0979", "CUSIP_4996", "CUSIP_1339", "CUSIP_9811", "CUSIP_8805", "CUSIP_3745", "CUSIP_1188", "CUSIP_9827", "CUSIP_2266", "CUSIP_7192", "CUSIP_8759", "CUSIP_8867", "CUSIP_6152", "CUSIP_3552", "CUSIP_4045", "CUSIP_6756", "CUSIP_8601", "CUSIP_7037", "CUSIP_6479", "CUSIP_0836", "CUSIP_4071", "CUSIP_4746", "CUSIP_9984", "CUSIP_3463", "CUSIP_6010", "CUSIP_1197", "CUSIP_5734", "CUSIP_6044", "CUSIP_6676", "CUSIP_4092", "CUSIP_0250", "CUSIP_0546", "CUSIP_1966", "CUSIP_4464", "CUSIP_4111", "CUSIP_5037", "CUSIP_4078", "CUSIP_2667", "CUSIP_8589", "CUSIP_0442", "CUSIP_5481", "CUSIP_1132", "CUSIP_2542", "CUSIP_8633", "CUSIP_2683", "CUSIP_0433", "CUSIP_9158", "CUSIP_4549", "CUSIP_7668", "CUSIP_8657", "CUSIP_0662", "CUSIP_2741", "CUSIP_7121", "CUSIP_8110", "CUSIP_1611", "CUSIP_4788", "CUSIP_0644", "CUSIP_0687", "CUSIP_7371", "CUSIP_9323", "CUSIP_7045", "CUSIP_2475", "CUSIP_5819", "CUSIP_9834", "CUSIP_8139", "CUSIP_2616", "CUSIP_8870", "CUSIP_1934", "CUSIP_9826", "CUSIP_7350", "CUSIP_3754", "CUSIP_1267", "CUSIP_7440", "CUSIP_4280", "CUSIP_2225", "CUSIP_8468", "CUSIP_8140", "CUSIP_0253", "CUSIP_8936", "CUSIP_8736"]}]}
```

, "CUSIP_4066", "CUSIP_6990", "CUSIP_7027", "CUSIP_7631", "CUSIP_9594", "CUSIP_5652", "CUSIP_7815", "CUSIP_4450", "CUSIP_4502", "CUSIP_7792", "CUSIP_3897", "CUSIP_8743", "CUSIP_8525", "CUSIP_0912", "CUSIP_0945", "CUSIP_6767", "CUSIP_8311", "CUSIP_0155", "CUSIP_3711", "CUSIP_3077", "CUSIP_5935", "CUSIP_2450", "CUSIP_2057", "CUSIP_1751", "CUSIP_7486", "CUSIP_3626", "CUSIP_3043", "CUSIP_7839", "CUSIP_1171", "CUSIP_5795", "CUSIP_9637", "CUSIP_9011", "CUSIP_4365", "CUSIP_0853", "CUSIP_9938", "CUSIP_7741", "CUSIP_5337", "CUSIP_2904", "CUSIP_7178", "CUSIP_6592", "CUSIP_8158", "CUSIP_2759", "CUSIP_9513", "CUSIP_1575", "CUSIP_7188", "CUSIP_1552", "CUSIP_3052", "CUSIP_1127", "CUSIP_5713", "CUSIP_8874", "CUSIP_5824", "CUSIP_3566", "CUSIP_9469", "CUSIP_4533", "CUSIP_6172", "CUSIP_4298", "CUSIP_9674", "CUSIP_6404", "CUSIP_1509", "CUSIP_6676", "CUSIP_0805", "CUSIP_8322", "CUSIP_2988", "CUSIP_5662", "CUSIP_5878", "CUSIP_2748", "CUSIP_5860", "CUSIP_4051", "CUSIP_1892", "CUSIP_5419", "CUSIP_9486", "CUSIP_8304", "CUSIP_3070", "CUSIP_8071", "CUSIP_6147", "CUSIP_8010", "CUSIP_4886", "CUSIP_6783", "CUSIP_6280", "CUSIP_6927", "CUSIP_0443", "CUSIP_1719", "CUSIP_0483", "CUSIP_1049", "CUSIP_5221", "CUSIP_9638", "CUSIP_9920", "CUSIP_7504", "CUSIP_9431", "CUSIP_2544", "CUSIP_4834", "CUSIP_4188", "CUSIP_4390", "CUSIP_4193", "CUSIP_6283", "CUSIP_1411", "CUSIP_4953", "CUSIP_7934", "CUSIP_6764", "CUSIP_5473", "CUSIP_9776", "CUSIP_6932", "CUSIP_0513", "CUSIP_1753", "CUSIP_5965", "CUSIP_6938", "CUSIP_9178", "CUSIP_5784", "CUSIP_3475", "CUSIP_4956", "CUSIP_6478", "CUSIP_4973", "CUSIP_0331", "CUSIP_0932", "CUSIP_2180", "CUSIP_2869", "CUSIP_4246", "CUSIP_2856", "CUSIP_7944", "CUSIP_0946", "CUSIP_6543", "CUSIP_6195", "CUSIP_5990", "CUSIP_5698", "CUSIP_0870", "CUSIP_1277", "CUSIP_5478", "CUSIP_9848", "CUSIP_9830", "CUSIP_0510", "CUSIP_4622", "CUSIP_1545", "CUSIP_2738", "CUSIP_2924", "CUSIP_3446", "CUSIP_3166", "CUSIP_0671", "CUSIP_9075", "CUSIP_9270", "CUSIP_3790", "CUSIP_3374", "CUSIP_6321", "CUSIP_9944", "CUSIP_0349", "CUSIP_4725", "CUSIP_2557", "CUSIP_6474", "CUSIP_4744", "CUSIP_7170", "CUSIP_4095", "CUSIP_4495", "CUSIP_6195", "CUSIP_4958", "CUSIP_0689", "CUSIP_3768", "CUSIP_1718", "CUSIP_0876", "CUSIP_5961", "CUSIP_1374", "CUSIP_2016", "CUSIP_1634", "CUSIP_3330", "CUSIP_0398", "CUSIP_1060", "CUSIP_2168", "CUSIP_1392", "CUSIP_9176", "CUSIP_1627", "CUSIP_1195", "CUSIP_9992", "CUSIP_1122", "CUSIP_8019", "CUSIP_5324", "CUSIP_5731", "CUSIP_7927", "CUSIP_0236", "CUSIP_2065", "CUSIP_5151", "CUSIP_6893", "CUSIP_2374", "CUSIP_9807", "CUSIP_7933", "CUSIP_8906", "CUSIP_8316", "CUSIP_5280", "CUSIP_5438", "CUSIP_2077", "CUSIP_1222", "CUSIP_2964", "CUSIP_6282", "CUSIP_4480", "CUSIP_5671", "CUSIP_4971", "CUSIP_8868", "CUSIP_9226", "CUSIP_3244", "CUSIP_0206", "CUSIP_5896", "CUSIP_6779", "CUSIP_8267", "CUSIP_1464", "CUSIP_1504", "CUSIP_7261", "CUSIP_6658", "CUSIP_3355", "CUSIP_8703", "CUSIP_7248", "CUSIP_7622", "CUSIP_9142", "CUSIP_8562", "CUSIP_8230", "CUSIP_7817", "CUSIP_1032", "CUSIP_5034", "CUSIP_9935", "CUSIP_5294", "CUSIP_0443", "CUSIP_4910", "CUSIP_0856", "CUSIP_3282", "CUSIP_6811", "CUSIP_3413", "CUSIP_1063", "CUSIP_8051", "CUSIP_9796", "CUSIP_7348", "CUSIP_2280", "CUSIP_4294", "CUSIP_6147", "CUSIP_4711", "CUSIP_3807", "CUSIP_7847", "CUSIP_9790", "CUSIP_3359", "CUSIP_6071", "CUSIP_6019", "CUSIP_5141", "CUSIP_8277", "CUSIP_3843", "CUSIP_4764", "CUSIP_1062", "CUSIP_8223", "CUSIP_0755", "CUSIP_6369", "CUSIP_7144", "CUSIP_7420", "CUSIP_3967", "CUSIP_7728", "CUSIP_3539", "CUSIP_4430", "CUSIP_0244", "CUSIP_0439", "CUSIP_0301", "CUSIP_6761", "CUSIP_8478", "CUSIP_0098", "CUSIP_0782", "CUSIP_1733", "CUSIP_2507", "CUSIP_3537", "CUSIP_4534", "CUSIP_1333", "CUSIP_2047", "CUSIP_2008", "CUSIP_5229", "CUSIP_7596", "CUSIP_3574", "CUSIP_0167", "CUSIP_6852", "CUSI

P_6294", "CUSIP_7006", "CUSIP_8887", "CUSIP_4169", "CUSIP_2381", "CUSIP_1790", "CUSIP_0431", "CUSIP_3233", "CUSIP_1432", "CUSIP_8303", "CUSIP_6892", "CUSIP_0370", "CUSIP_7448", "CUSIP_9102", "CUSIP_7461", "CUSIP_8817", "CUSIP_3728", "CUSIP_1384", "CUSIP_6107", "CUSIP_2683", "CUSIP_6513", "CUSIP_0643", "CUSIP_3673", "CUSIP_9398", "CUSIP_3647", "CUSIP_8853", "CUSIP_5095", "CUSIP_1213", "CUSIP_3029", "CUSIP_1739", "CUSIP_4689", "CUSIP_5655", "CUSIP_5874", "CUSIP_8229", "CUSIP_5109", "CUSIP_5964", "CUSIP_3726", "CUSIP_3138", "CUSIP_3168", "CUSIP_4816", "CUSIP_2500", "CUSIP_0549", "CUSIP_6568", "CUSIP_1505", "CUSIP_4911", "CUSIP_6531", "CUSIP_7684", "CUSIP_3034", "CUSIP_7578", "CUSIP_1071", "CUSIP_9598", "CUSIP_6500", "CUSIP_6803", "CUSIP_6062", "CUSIP_3748", "CUSIP_8042", "CUSIP_2691", "CUSIP_1624", "CUSIP_9483", "CUSIP_3591", "CUSIP_5422", "CUSIP_3350", "CUSIP_2641", "CUSIP_1405", "CUSIP_1702", "CUSIP_5825", "CUSIP_2617", "CUSIP_1506", "CUSIP_8754", "CUSIP_4915", "CUSIP_5046", "CUSIP_9449", "CUSIP_4695", "CUSIP_3154", "CUSIP_8904", "CUSIP_5703", "CUSIP_5084", "CUSIP_2559", "CUSIP_4296", "CUSIP_2862", "CUSIP_9780", "CUSIP_9740", "CUSIP_1364", "CUSIP_0374", "CUSIP_0484", "CUSIP_1780", "CUSIP_7094", "CUSIP_4871", "CUSIP_1291", "CUSIP_6042", "CUSIP_4584", "CUSIP_2751", "CUSIP_8634", "CUSIP_8149", "CUSIP_3492", "*"]}, "lendingIdGroups.values": { "\$in": ["LEND_001222", "LEND_425253", "LEND_620131", "LEND_645419", "LEND_611199", "LEND_642149", "LEND_906888", "LEND_894725", "LEND_004253", "LEND_244817", "LEND_989132", "LEND_643827", "LEND_942947", "LEND_685501", "LEND_048063", "LEND_090891", "LEND_559524", "LEND_692299", "LEND_906996", "LEND_253503", "LEND_118920", "LEND_999494", "LEND_807540", "LEND_635851", "LEND_759500", "LEND_984279", "LEND_316187", "LEND_902909", "LEND_778276", "LEND_380866", "LEND_479084", "LEND_329044", "LEND_568460", "LEND_158159", "LEND_778425", "LEND_805779", "LEND_562485", "LEND_602178", "LEND_416789", "LEND_946529", "LEND_902348", "LEND_429893", "LEND_388693", "LEND_653046", "LEND_596084", "LEND_979877", "LEND_645481", "LEND_167223", "LEND_193733", "LEND_238610", "LEND_048342", "LEND_480336", "LEND_791438", "LEND_898359", "LEND_808459", "LEND_846860", "LEND_779467", "LEND_034055", "LEND_697682", "LEND_018784", "LEND_120619", "LEND_728699", "LEND_430344", "LEND_551951", "LEND_963959", "LEND_988941", "LEND_288548", "LEND_815997", "LEND_231403", "LEND_312869", "LEND_249629", "LEND_456496", "LEND_697489", "LEND_059331", "LEND_505725", "LEND_524909", "LEND_916304", "LEND_117014", "LEND_169826", "LEND_143158", "LEND_251273", "LEND_942913", "LEND_839647", "LEND_408470", "LEND_735849", "LEND_597492", "LEND_850408", "LEND_624781", "LEND_969591", "LEND_371440", "LEND_879320", "LEND_929185", "LEND_354476", "LEND_378806", "LEND_358900", "LEND_632328", "LEND_282744", "LEND_523539", "LEND_622162", "LEND_109406", "LEND_516910", "LEND_442907", "LEND_427237", "LEND_294884", "LEND_301178", "LEND_287451", "LEND_361562", "LEND_219156", "LEND_781698", "LEND_286921", "LEND_967289", "LEND_922817", "LEND_724322", "LEND_594721", "LEND_616946", "LEND_427943", "LEND_476675", "LEND_785630", "LEND_428195", "LEND_179534", "LEND_135158", "LEND_810607", "LEND_339048", "LEND_484792", "LEND_354832", "LEND_652380", "LEND_796302", "LEND_479638", "LEND_710155", "LEND_769962", "LEND_375137", "LEND_907886", "LEND_244697", "LEND_317268", "LEND_048599", "LEND_806961", "LEND_894862", "LEND_480128", "LEND_906448", "LEND_595145", "LEND_273622", "LEND_651110", "LEND_883842", "LEND_800611", "LEND_544565", "LEND_045565", "LEND_338004", "LEND_106084", "LEND_466818", "LEND_420766", "LEND_435594", "LEND_120485", "LEND_989878", "LEND_731320", "LEND_328695", "LEND_252441", "LEND_193041", "LEND_421417", "LEND_945716", "LEND_15

```
5341", "LEND_340218", "LEND_798391", "LEND_537003", "LEND_913181", "LEND_036450", "LEND_582396", "LEND_350673", "LEND_348854", "LEND_210405", "LEND_980933", "LEND_355454", "LEND_283979", "LEND_917226", "LEND_956034", "LEND_292910", "LEND_604360", "LEND_856614", "LEND_686805", "LEND_924133", "LEND_405799", "LEND_794247", "LEND_799722", "LEND_552431", "LEND_272969", "LEND_614414", "LEND_625787", "LEND_865934", "LEND_039275", "LEND_955716", "LEND_945859", "LEND_959391", "LEND_643456", "LEND_717720", "LEND_951020", "LEND_886613", "LEND_833263", "LEND_263058", "LEND_239257", "LEND_475069", "LEND_528530", "LEND_306545", "LEND_329853", "LEND_654636", "LEND_279359", "LEND_940352", "LEND_672751", "LEND_083774", "LEND_673475", "LEND_414833", "LEND_719128", "LEND_043100", "LEND_886444", "LEND_596623", "LEND_340694", "LEND_755415", "LEND_963351", "LEND_087555", "LEND_484768", "LEND_855137", "LEND_939971", "LEND_948716", "LEND_877481", "LEND_020873", "LEND_385247", "LEND_349378", "LEND_690861", "LEND_163465", "LEND_340475", "LEND_126197", "LEND_139851", "LEND_066393", "LEND_033134", "LEND_949194", "LEND_833607", "LEND_894650", "LEND_651586", "LEND_893559", "LEND_987265", "LEND_146102", "LEND_043773", "LEND_936712", "LEND_977993", "LEND_505554", "LEND_136702", "LEND_671738", "LEND_918130", "LEND_115502", "LEND_707373", "LEND_056359", "LEND_823871", "LEND_357635", "LEND_812780"]}}}}}}}, "planSummary": "IXSCAN { meta.isActive: 1, lendingIdGroups.values: 1 }", "planningTimeMicros": 7033, "numYields": 0, "planCacheShapeHash": "2D899393", "queryHash": "2D899393", "planCacheKey": "F1CE8BF5", "queryFramework": "classic", "reslen": 153911, "locks": {"Global": {"acquireCount": {"r": 1}}}, "readConcern": {"level": "local", "provenance": "implicitDefault"}, "storage": {}, "remote": "127.0.0.1:53601", "protocol": "op_msg", "queues": {"ingress": {"admissions": 1}, "execution": {"admissions": 2}}, "workingMillis": 8, "durationMillis": 8, "truncated": {"command": {"truncated": {"explain": {"truncated": {"pipeline": {"truncated": {"0": {"truncated": {"$match": {"truncated": {"lendingIdGroups.values": {"truncated": {"$in": {"truncated": {"252": {"type": "string", "size": 16}}, "omitted": 248}}}}}}}}}, "omitted": 1}}, "omitted": 5}}, "size": {"command": 20797}}
```